

Maximize the Rightmost Digit: Gray Codes for Restricted Growth Strings

Yuan (Friedrich) Qiu¹, Joe Sawada², and Aaron Williams¹[0000–0001–6816–4368]

Williams College, Williamstown MA 01267, USA
<https://csci.williams.edu/people/faculty/aaron-williams/>
 {yq1, aaron.williams}@williams.edu

Abstract. The term *restricted growth string* typically refers to strings of non-negative integers $a_1 a_2 \cdots a_n$ (with $a_1 = 0$) in which the next symbol is at most one more than the maximum of the previous symbols: $0 \leq a_i \leq \max(a_1 \cdots a_{i-1}) + 1$ for $2 \leq i \leq n$. These strings are counted by the Bell numbers $\mathcal{B}_n$ (OEIS A000110) and encode set partitions. Kerr showed that the following greedy algorithm generates a Gray code starting from 0^n : greedily maximize the rightmost symbol that creates a new string (e.g., 000, 001, 011, 012, 010 for $n = 3$). This turns out to be a special case of a more general result for **e**-restricted growth functions Mansour, Nassar, and Vajnovszki's, although those authors did not describe their order greedily. Here we generalize these results to a much broader class of restricted growth strings: $0 \leq a_1 \leq s-1$ and $0 \leq a_i \leq f(a_1 a_2 \cdots a_{i-1}) + c_i$ where f is any function with $f \geq 0$ and $c_i \geq 1$ are constants for each digit. In each case, the orders change a single digit by -1 or -2 (cyclically). Special cases include the binary reflected Gray code ($s = 2$, $f = 0$, $c = 1$) and the aforementioned orders for $\mathbf{c} = \mathbf{e}$. We also introduce a new type of restricted growth string counted by the k -Catalan numbers and provide a loopless algorithm for generating them.

Keywords: restricted growth strings · Bell numbers · set partitions · Catalan numbers · Gray codes · greedy Gray codes.

1 Introduction

In this paper, we are interested in efficiently ordering and generating various types of restricted growth strings. We begin by describing two of the most common types of restricted growth strings, along with their applications.

1.1 Bell and Catalan Strings

The term *restricted growth string* is often defined as a string of integers (called *digits*) of the form $a_1 a_2 \cdots a_n$ that satisfies the following conditions,

$$a_1 = 0 \text{ and } 0 \leq a_i \leq \max(a_1 a_2 \cdots a_{i-1}) + 1 \text{ for } 2 \leq i \leq n. \quad (1)$$

In other words, the first digit is 0, and each subsequent digit is at least 0 and at most one more than the maximum of the previous digits. The number of restricted growth strings of length $n \geq 0$ is the n^{th} Bell number $\mathcal{B}_n$.

$$1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975, \dots \quad \text{OEIS A000110 [8]}. \quad (2)$$

Since they are enumerated by the Bell numbers, we refer to this type of restricted growth string as *Bell strings*. Bell strings provide a convenient representation for the set partitions of $[n] = \{1, 2, \dots, n\}$, which are also Bell objects. The standard bijection puts i into the a_i^{th} part, as shown below for $n \leq 3$ [22].

0	00	01	000	001	010	011	012
{1}	{1,2}	{1},{2}	{1,2,3}	{1,2},{3}	{1,3},{2}	{1},{2,3}	{1},{2},{3}

Note that a small change in a set partition can lead to a large change in its Bell string. For example, the set partition $\{1, 2\}, \{3\}, \{4\}, \dots, \{n\}$ corresponds to the Bell string $00123 \cdots (n-2)$. If we move the 2 into its own subset to create the set partition of singletons, then the corresponding Bell string becomes $0123 \cdots (n-1)$ (i.e., all digits change except the leading 0). On the other hand, changing a single digit in a Bell string always corresponds to moving a single value in its set partition. For this reason, when designing efficient orders of set partitions it can be preferable to instead work with Bell strings.

Perhaps the most well-known ordering of set partitions was created by Knuth and presented by Kaye [9]. Later work by Ruskey and Savage [19] adapted the approach to Bell strings. A student project by Kerr [10] provided an alternate ordering of Bell strings (see [17]) that uses a greedy approach [25]. This paper generalizes Kerr's result from Bell strings to other restricted growth strings.

Another interesting type of restricted growth string is obtained by modifying (1)

$$a_1 = 0 \text{ and } 0 \leq a_i \leq a_{i-1} + 1 \text{ for } 2 \leq i \leq n. \quad (3)$$

The number of *Catalan strings* of length $n \geq 0$ is the n^{th} Catalan number $\mathcal{C}_n$

$$1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \dots \quad \text{OEIS A000108.} \quad (4)$$

We let $\mathbf{B}(n)$ and $\mathbf{C}(n)$ be the sets of Bell and Catalan strings of length n , respectively. Figure 3 has lists of $\mathbf{B}(n)$ and $\mathbf{C}(n)$ for $n \leq 4$. In particular, the reader can confirm the following: $\mathbf{B}(4) \setminus \mathbf{C}(4) = \{0102\}$.

Catalan strings provide an alternate representation for the large number of other Catalan objects counted by $\mathcal{C}_n$ [23]. We'll also provide a simple generalization to k -Catalan strings $\mathbf{C}_k(n)$ in Section 2. There are several other types of strings counted by Catalan and k -Catalan numbers (e.g., see [26]).

1.2 Generalized Restricted Growth Strings

Although the term restricted growth string often refers specifically to Bell strings, it is also used much more broadly in the literature. Here we consider a generalization that allows for flexibility in the first digit, the function applied to the previous digits, and the constant added to each digit. Formally, an $(s, f, \mathbf{c})$ -restricted growth string is a string of integers of the form $a_1 a_2 \cdots a_n$ satisfying

$$0 \leq a_1 \leq s - 1 \text{ and } 0 \leq a_i \leq f(a_1 a_2 \cdots a_{i-1}) + c_i \text{ for } 2 \leq i \leq n \quad (5)$$

with $s \geq 1$, $f \geq 0$, and $c \geq 1$. In other words, the *starting digit* a_1 has s possible values $0 \leq a_1 \leq s$, while each subsequent digit a_i is a non-negative integer

limited by the sum of a *function* f that maps the previous digits $a_1 a_2 \cdots a_{i-1}$ to a non-negative integer, and a positive integer *constant* c_i that depends only on the index i . (For notational convenience, we write $c = w$ and $f = w$ when $c_i = w$ and $f(a_1 a_2 \cdots a_{i-1}) = w$ for all $2 \leq i \leq n$, respectively.)

This generalization captures a wide variety of previously studied strings, including those discussed in Section 1.1. Binary strings are obtained with $s = 2$, $f = 0$, and $c = 1$, while mixed-radix strings with bases $b_1, b_2, \dots, b_n$ (see Section 3.3) are obtained with $s = b_1$, $f = 0$, and $c_i = b_i - 1$ for $2 \leq i \leq n$. Additional special cases of $(s, f, \mathbf{c})$ -restricted growth strings and related previous results include the following.

- *Restricted growth tails* [19] are modeled by using different values of s .
- *Restricted growth strings of order d* are obtained by replacing $+1$ in (1) with $+(d-1)$. More broadly, the *e-restricted growth functions* put forward by Mansour, Nassar and Vajnovszki [14] are modeled by our constants $\mathbf{c}$. Their paper also uses the greedy max-right order, although it does not identify it as such. In particular, their $\text{succ}_{1,m}$ and $\text{succ}_{2,m}$ lists mirror our g_0 and g_1 expansions (see Section 4).
- *Prefix statistics* (e.g., ascents) were considered by Mansour and Vajnovszki [15] and Sabri and Vajnovszki [20] and are modeled by our functions f . By carefully choosing the ‘statistic’ and ignoring the first digit of 0, the *st*-restricted strings concept in [15] can be used to model $(s, f, \mathbf{c})$ -RGS strings.
- *K-increment strings* replace $+1$ with $+i$ in (3) and are modeled by $\mathbf{c}$. Arndt efficiently generates these strings and other restricted growth strings [1].

1.3 Goals and Results

We are interested in creating Gray codes for restricted growth strings. That is, we want to list these sets so that consecutive strings differ in a small constant amount. Furthermore, we want to generate these lists efficiently. In this context, *constant amortized time (CAT)* and *loopless* algorithms generate successive strings in amortized and worst-case $O(1)$ -time, respectively.

An initial roadblock is that Bell strings do not have a ± 1 Gray code [18]. In other words, it is not possible to order the strings in an arbitrary $\mathbf{B}(n)$ so that consecutive strings differ in only one digit, and such that the differing values are within ± 1 of each other. However, Ehrlich [4] constructed a Gray code for $\mathbf{B}(n)$ in which a single digit changes by ± 1 when considered cyclically¹ and provided a loopless implementation. On the other hand, Ruskey [18] created a CAT algorithm that allows ± 1 and ± 2 non-cyclically². Li and Sawada provided a Gray code for $\mathbf{B}(n)$ as part of their *reflectable languages* framework [13], and their special values $x = 0$ and $y = 1$ arise naturally in our results.

¹ A flexible notion of cyclicity is used: $a_i = 0 \leftrightarrow a_i = m$ and $a_i = 0 \leftrightarrow a_i = m + 1$ are allowed when $m = \max(a_1 a_2 \cdots a_{i-1})$.

² This order was also mentioned in a paper by Ruskey and Savage [19], however, the two descriptions are not equivalent.

Our goal is to present an approach to generating restricted growth string Gray codes with the following benefits:

- (a) The approach is very easy to describe.
- (b) The approach generalizes previous results.
- (c) The approach works for all $(s, f, \mathbf{c})$ -restricted growth strings.
- (d) The approach leads to loopless generation algorithms.

We reach our goals using an approach that can be summarized in one sentence: start a list with 0^n then repeatedly extend it to a new string by greedily changing the rightmost digit to the maximum possible value. We refer to this approach as the *max-right algorithm*, and we note that “allowable” depends on which type of string is being generated. As we’ll see, successive strings in the resulting *max-right orders* differ in a single digit by -1 or -2 (cyclically). In particular, the change from 0 to the maximum possible value is -1 taken cyclically, and 1 to the maximum value is -2 taken cyclically. Moreover, we provide loopless implementations and applications for $\mathbf{B}(n)$ and $\mathbf{C}_k(n)$. We also obtain the ‘original’ Gray code for n -bit binary strings using $s = 2$, $f = 0$, and $c = 1$.

1.4 Outline

Section 2 introduces k -Catalan strings and proves that they are an example of $(s, f, \mathbf{c})$ -restricted growth strings. Section 3 discusses Gray codes and combinatorial generation. Section 4 provides our Gray codes for $(s, f, \mathbf{c})$ -restricted growth strings. Section 5 provides new loopless algorithms for mixed-radix, k -Catalan, and Bell strings. The appendix includes additional figures and Python implementations of our algorithms.

2 k -Catalan Strings

In this section we provide a natural generalizations of Catalan strings, and prove that they are an example of $(s, f, \mathbf{c})$ -restricted growth strings. Our generalization of Catalan strings comes from replacing $+1$ with $+(k-1)$ in (3) as follows.

$$a_1 = 0 \text{ and } 0 \leq a_i \leq a_{i-1} + (k-1) \text{ for } 2 \leq i \leq n. \quad (6)$$

We refer to the resulting strings as *k-Catalan strings* and let $\mathbf{C}_k(n)$ be the set of length n . For example, when $n = 3$ and $k = 3$ we have

$$\mathbf{C}_3(3) = \{000, 001, 002, 010, 011, 012, 013, 020, 021, 022, 023, 024\} \quad (7)$$

and these $|\mathbf{C}_3(3)| = 12$ strings are in bijection with the ternary trees with 3 internal nodes. The Catalan strings $\mathbf{C}(n)$ defined in (3) are obtained from (6) with $k = 2$, and Catalan numbers are also known as 2-Catalan numbers $\mathcal{C}_n$. More broadly, $\mathcal{C}_{k,n}$ denotes the n th k -Catalan number, and the OEIS sequences A000108, A001764, A002293, A002294, A002295 are for $2 \leq k \leq 6$.

Theorem 1. $|\mathbf{C}_k(n)| = \mathcal{C}_{k,n}$ for all $n \geq 0$ and $k \geq 2$.

Proof. We prove that the members of $\mathbf{C}_k(n)$ are in bijective correspondence with the k -ary trees with n internal nodes, which are known to be counted by $\mathcal{C}_{k,n}$. Note that there is a single k -ary tree with one internal node and that $\mathbf{C}_k(n) = \{0\}$, so the result is true for $n = 1$ and all $k \geq 2$. Suppose the result is true for $n = t$ and all $k \geq 2$. Now we consider how the strings and trees can be extended to $n = t + 1$. Every string in $\mathbf{C}_k(t)$ that ends with the digit d is the prefix of $d + (k - 1)$ strings in $\mathbf{C}_k(t + 1)$. Next consider a k -ary tree with t internal nodes and label its nodes according to a preorder traversal. Consider the location of the node x that is last in preorder (i.e., it has label t). To extend this tree without changing the preorder traversal we can either add the new node as a child of x or as the right-child of any node on the path from the root to x that doesn't have a right-child. In this way, if x was in the d th rightmost leaf in the tree, then the added node can be in any of $d + (k - 1)$ positions.

Theorem 2. The set of k -Catalan strings $\mathbf{C}_k(n)$ are an example of $(s, f, \mathbf{c})$ -restricted growth strings.

Proof. We claim this is true from $s = 1$, $f(a_1 a_2 \cdots a_{i-1}) = a_{i-1}$, and $c_i = k - 1$ for all $2 \leq i \leq n$. This follows from (5) as these choices force $0 \leq a_1 = s - 1 = 0$ (i.e., $a_1 = 0$) and the following bound for $2 \leq i \leq n$ that matches (6),

$$0 \leq a_i \leq f(a_1 a_2 \cdots a_{i-1}) + c_i = a_{i-1} + (k - 1). \quad (8)$$

(Similarly, $\mathbf{C}_k(n)$ are st-restricted strings [15] using the statistic $a_{i-1} + (k - 1)$.)

3 Gray Codes and Combinatorial Generation

The term *Gray code* refers to an exhaustive list of some combinatorial object (parameterized by size) in which successive objects differ in some (small) way. They are named after the famous order of n -bit binary strings with Hamming distance one (i.e., a single bit's value is complemented or *flipped*) in Frank Gray's 1954 patent titled *Pulse Code Communication* [5]. The order is referred to as the *binary reflected Gray code (brgc)* and it appears below for $n = 3$, with overlines denoting the bit that changes to create the next string.

$$\mathbf{brgc}(3) = 00\overline{0}, 0\overline{0}1, 01\overline{1}, \overline{0}10, 11\overline{0}, 1\overline{1}1, 10\overline{1}, 100 \quad (9)$$

$$\mathbf{plain}(3) = 1\overleftarrow{2}3, \overleftarrow{1}32, 3\overleftarrow{1}2, \overrightarrow{3}21, \overrightarrow{2}31, 213 \quad (10)$$

Plain changes predates the 'first' Gray code by hundred years and is illustrated for $n = 3$ in (10). In this order, consecutive permutations of $[n] = \{1, 2, \dots, n\}$ differ by a *swap* (i.e., adjacent entries are transposed) with the arrows in (10) showing a larger value "jumping over" a smaller value. The order was performed by bell-ringers in the 1600s [24], and is known as the *Steinhaus-Johnson-Trotter algorithm* due to multiple rediscoveries in the mid-20th century.

Traditionally, Gray codes have been discovered and described recursively. For example, note that **brgc**(3) is obtained from two copies of **brgc**(2) = 00, 01, 11, 10 by prefixing 0 to the strings in the first copy, and 1 to the strings in the second copy which is first *reflected* to 10, 11, 01, 00. Similarly, **plain**(3) is obtained from **plain**(2) = 12, 21 by sweeping 3 from right-to-left through 12 then left-to-right through 21. The first approach uses *global recursion* since **brgc**(n) is created from full copies of **brgc**($n-1$), while the second uses *local recursion* since **plain**(n) expands individual objects in **plain**($n-1$).

Countless Gray codes have been constructed over the years. Academic surveys have been written by Savage [21] and more recently Mütze [17], with Ruskey [18] and Knuth [11] devoting extensive textbook coverage to the subject. In fact, one of the issues facing this research area is the sheer breadth of results and the recursive ‘tricks’ that have been used to obtain them. For an interactive introduction to the area, we recommend the *combinatorial object server* combos.org.

3.1 Greedy Gray Code Algorithm

This decade has seen the introduction of the *greedy Gray code algorithm* [25]. The algorithm eschews recursive schemes to focus on a simple idea: build an order one object at a time by prioritizing the possible changes. For example, the BRGC can be constructed starting from 0^n (where exponentiation denotes concatenation) by greedily flipping the rightmost possible bit. Similarly, plain change starts at $12 \cdots n$ and then greedily swaps the largest possible value³. To clarify these descriptions, consider the partial orders below.

$$\mathbf{brgc}(3) = 00\bar{0}, 0\bar{0}1, 01\bar{1}, 010, \dots? \quad (11)$$

$$\mathbf{plain}(3) = 1\bar{2}3, \bar{1}32, 312, \dots? \quad (12)$$

Which binary string should follow 010? Flipping the rightmost bit would give $01\bar{0} = 011$, but that string is already in (11). Similarly, flipping the middle bit would also result in a repeated string $0\bar{1}0 = 000$. However, flipping the leftmost bit gives a new string $\bar{0}10 = 110$, so it is the next string in the order. Likewise, in (12) we cannot swap 3 to the right as it would recreate $3\bar{1}2 = 132$, nor can we swap it to the left as it is already in the leftmost position. Thus, our highest priority change is to swap the next largest value 2 to the left to create $3\bar{1}2 = 321$.

These two greedy descriptions are not efficient in the sense of *combinatorial generation*, which is focused on efficiently generating exhaustive lists of combinatorial objects. More specifically, both algorithms require an exponential amount of space to determine if a specific change creates a new string or not. However, it is often possible to find an alternate description of a greedily defined order, such as the recursive descriptions of **brgc**(n) and **plain**(n) discussed earlier.

The greedy Gray code algorithm has also proven to be extremely adept at uncovering new results. In particular, the greedy description of plain change

³ The latter description has the potential to be ambiguous—should a value be swapped to the left or right?—but in practice there is always a unique choice.

order was the impetus for the *permutation language* series [6] [7] [16] [3] [2]. Similarly, our new results can be seen as a somewhat unexpected generalization of the binary reflected Gray code.

3.2 Four Greedy Definitions of the Binary Reflected Gray Code

Here we provide four different greedy algorithms for generating the binary reflected Gray code starting from 0^n . The first approach was previously discussed, and it should be clear that the other three approaches produce identical results.

1. Greedily complement the rightmost bit.
2. Greedily increment or decrement the rightmost bit.
3. Greedily increment the rightmost bit cyclically modulo 2.
4. Greedily set the rightmost bit to the maximum possible value.

Figure 1 illustrates the four interpretations for **brgc**(4). In the figure, we use the symbols $\bar{}$ for complement, ± 1 for increment / decrement, $\oplus$ for cyclic increment, and max for maximum possible value. While the four algorithms give the same order for binary strings, we'll see that the last three produce different orders for other sets of strings; we henceforth ignore the first algorithm as complements cannot be applied to non-binary digits. Eventually, we'll see that max approach has a particular advantage for restricted growth strings, as observed in [14] [13].

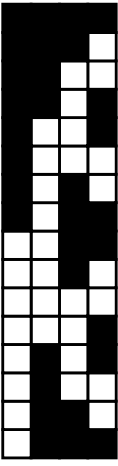
brgc (4)	$b_4b_3b_2b_1$	$\bar{}$	$\pm$	$\oplus$	max
	0000	$\bar{1}$	$+1$	$\oplus_1$	max ₁
	0001	$\bar{2}$	$+2$	$\oplus_2$	max ₂
	0011	$\bar{1}$	$+1$	$\oplus_1$	max ₁
	0010	$\bar{3}$	-3	$\oplus_3$	max ₃
	0110	$\bar{1}$	$+1$	$\oplus_1$	max ₁
	0111	$\bar{2}$	-2	$\oplus_2$	max ₂
	0101	$\bar{1}$	-1	$\oplus_1$	max ₁
	0100	$\bar{4}$	-4	$\oplus_4$	max ₄
	1100	$\bar{1}$	$+1$	$\oplus_1$	max ₁
	1101	$\bar{2}$	$+2$	$\oplus_2$	max ₂
	1111	$\bar{1}$	-1	$\oplus_1$	max ₁
	1110	$\bar{3}$	-3	$\oplus_3$	max ₃
	1010	$\bar{1}$	$+1$	$\oplus_1$	max ₁
	1011	$\bar{2}$	-2	$\oplus_2$	max ₂
	1001	$\bar{1}$	-1	$\oplus_1$	max ₁
	1000				

Fig. 1: Four greedy interpretations of **brgc**(4).

3.3 Three Greedy Gray Codes for Mixed-Radix Strings

Let $b_1, b_2, \dots, b_n$ be a list of positive integers called *bases*. A *mixed-radix string* with these bases is any $a_1a_2 \dots a_n$ with $1 \leq a_i \leq b_i - 1$ for all i . In other words,

each b_i provides the number of values that the i^{th} digit can hold. Figure 2 illustrates how three of the greedy approaches mentioned in Section 3.2 generate Gray codes for the strings with bases 1, 2, 3, 4. In each case, the reader's attention should be drawn to the different patterns created in the rightmost digit.

- When using increments and decrements (Figure 2a) the rightmost digit pings-pongs back-and-forth: 0, 1, 2, 3, 3, 2, 1, 0, 0, 1, 2, 3, 3, 2, 1, 0, ... reflectively.
- When using cyclic increments (Figure 2b) the rightmost digit's starting value climbs on each block 0, 1, 2, 3, 3, 0, 1, 2, 2, 3, 0, 1, 1, 2, 3, 0, 0, 1, 2, 3, 3, 0, 1, 2.
- When using maximization (Figure 2c) the rightmost digit alternately starts with 0 and ends with 1 or vice versa 0, 3, 2, 1, 1, 3, 2, 0, 0, 3, 2, 1, 1, 3, 2, 0, ...

The third pattern is quite useful in the context of restricted growth strings. This is because lower values are less likely to exceed their digit's upper bound, so having 0 and 1 as the first and last values (or vice versa) allows the greedy algorithm to uncover safer forms of recursion.

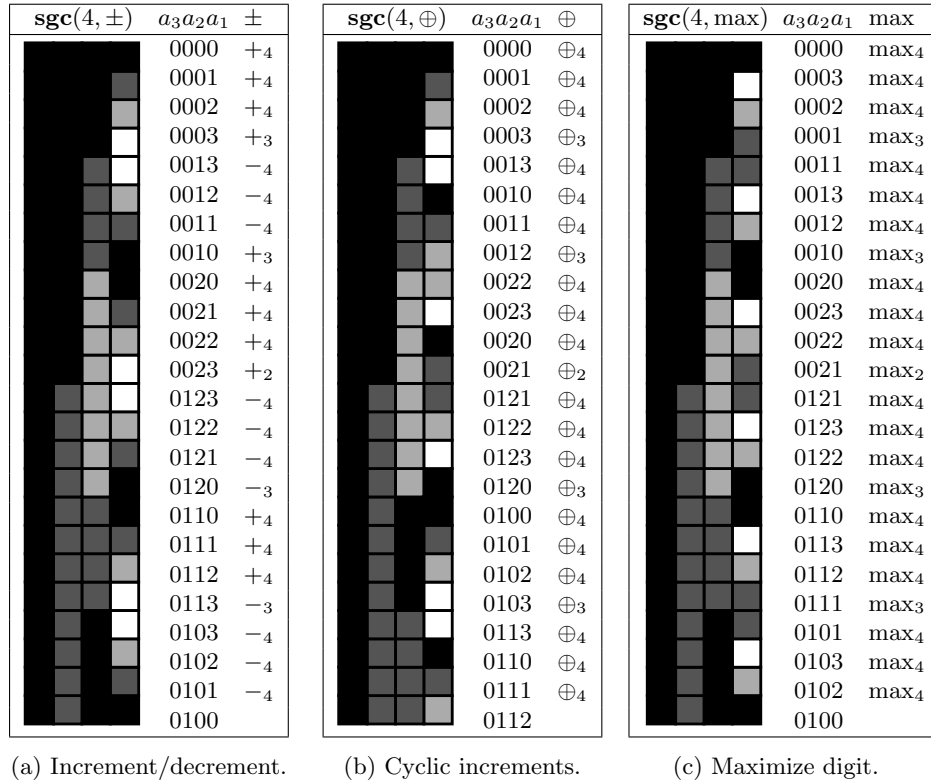


Fig. 2: Three greedy Gray codes for mixed-radix strings with bases 1, 2, 3, 4. Each one greedily applies an operation (or operations) to the rightmost possible digit.

4 Main Result

Theorem 3. *The greedy max-right algorithm generates all $(s, f, \mathbf{c})$ -restricted growth strings of length n , and successive strings differ by -1 or -2 (cyclically) in one digit.*

Proof. Recall from (5) that $a_1 a_2 \cdots a_n$ is an $(s, f, \mathbf{c})$ -restricted growth string if

$$0 \leq a_1 \leq s-1 \text{ and } 0 \leq a_i \leq f(a_1 a_2 \cdots a_{i-1}) + c_i \text{ for } 2 \leq i \leq n$$

with $s \geq 1$, $f \geq 0$, and $c \geq 1$. We prove the theorem by induction on $n \geq 1$.

For the base case of $n = 1$, notice that the conditions reduce to $0 \leq a_1 \leq s-1$. Therefore, the greedy max-right algorithm produces the list $0, s-1, s-2, \dots, 1$.

Assume that the result holds for all valid choices and $n = k$. Now consider a specific choice of s , f , and $\mathbf{c}$ with $n = k+1$. Let f' and $\mathbf{c}'$ be the restrictions of f and $\mathbf{c}$ to $n = k$, respectively. By induction, the greedy max-right algorithm creates a Gray code for the $(s, f', \mathbf{c}')$ strings of length k . Let this Gray code be $x_1, x_2, \dots, x_p$ where p is the number of such strings. Now consider the greedy max-right algorithm for the $(s, f, \mathbf{c})$ strings of length $k+1$. We claim that the algorithm will generate the strings in the following order,

$$\begin{aligned} &g_0(x_1), g_1(x_2), g_0(x_1), g_1(x_2), \dots, g_0(x_p) \text{ if } p \text{ is odd} \\ &g_0(x_1), g_1(x_2), g_0(x_1), g_1(x_2), \dots, g_1(x_p) \text{ if } p \text{ is even} \end{aligned} \quad (13)$$

where the g_0 and g_1 functions expand each x_i string of length k into a sublist of strings of length $k+1$ in a manner described below. Towards these definitions, let $x_i = a_1 a_2 \cdots a_k$. Therefore, $m = f(x_i) + c_{k+1}$ is the maximum value such that $x_i \cdot m$ is a $(s, f, \mathbf{c})$ string. We also know that $m \geq 1$ due to the conditions that $f \geq 0$ and $\mathbf{c} \geq 1$. The two expansions of x_i are now defined as follows,

$$\begin{aligned} g_0(x_i) &= x_i \cdot 0, x_i \cdot m, x_i \cdot (m-1), \dots, x_i \cdot 2, x_i \cdot 1 \\ g_1(x_i) &= x_i \cdot 1, x_i \cdot m, x_i \cdot (m-1), \dots, x_i \cdot 2, x_i \cdot 0. \end{aligned} \quad (14)$$

In both cases, the expansion sets the last digit to the maximum value m and then repeatedly decrements it. The difference between the two expansions is that the last digit starts at 0 and ends at 1 in the g_0 expansion, and vice versa in the g_1 expansion. To complete the proof we need to argue the following points:

- The greedy max-right algorithm does indeed generate the list in (13).
- The list in (13) includes all $(s, f, \mathbf{c})$ strings of length $k+1$.
- Successive strings in (13) differ in a single digit by -1 or -2 (cyclically).

To prove the first point, note that the greedy max-right algorithm prefers to change the rightmost digit to the maximum possible value that results in a new string. Therefore, if $x_i \cdot 0$ is the first string to be created with prefix x_i , then the algorithm will proceed by generating the list $g_0(x_i)$. Similarly, if $x_i \cdot 1$ is the first string to be created with prefix x_i , then the algorithm will proceed by generating the list $g_1(x_i)$. In both cases, all of the strings with prefix x_i

are generated in succession. Therefore, when the expansion of x_i is completed, the algorithm will then set the rightmost possible digit in x_i to the maximum possible value. By induction, this means that the prefix x_i will be replaced by x_{i+1} by the next change. Finally, note that the sublist $g_0(x_i)$ ends with $x_i \cdot 1$, so the aforementioned change will result in $x_{i+1} \cdot 1$, which is the first string of $g_1(x_{i+1})$. Similarly, the sublist $g_2(x_i)$ ends with $x_i \cdot 0$, so the aforementioned change will result in $x_{i+1} \cdot 0$, which is the first string of $g_0(x_{i+1})$. Hence, the expanded sublists alternate as per (13).

The second point follows from the fact that a digit's valid values are between 0 and m inclusively. The third point follows from (14) and induction. $\square$

5 Loopless Algorithms

In this section we provide loopless algorithms for generating multi-radix strings, k -Catalan strings, and Bell strings according to our max-right Gray codes.

5.1 Loopless Mixed-Radix Algorithms

The **MixedRadix** function in Algorithm 1 provides the well-known loopless algorithm for generating a mixed-radix Gray code using increment/decrement (i.e., ± 1) changes (see Knuth's description in [12]). The modified function **MixedRadixMax** in Algorithm 1 instead implements our mixed-radix Gray code using max changes. In this implementation, the s values keep track of the starting value of the corresponding digit: 0 or 1 (as per Section 4).

5.2 Loopless k -Catalan Strings

Our loopless implementation of our max-right k -Catalan Gray code is based on **MixedRadixMax**. One major difference is that the bases for each digit are not provided as inputs. Instead, they are computed as we generate the Gray code: the base of any position is the previous position's value plus $k - 1$.

Theorem 4. *CatalanStrings in Algorithm 2 looplessly generates the max-right Gray code for k -Catalan strings.*

5.3 Loopless Bell Strings

In our loopless implementation of the max-right Gray code for Bell strings, the concept of bases is not directly used, since computing the base of any digit (which is the maximum of previous digits plus 1) is a worst-case $\Theta(n)$ operation. Instead, we store the first positions of digits equal to successively larger values ≥ 2 (i.e., 2, 3, 4, ...) in a stack **S**. If the stack is non-empty, then its size allows us to determine a digit's maximum value. If the stack is empty, then the maximum is typically 2, since the earlier digits are comprised of 0s and 1s by (14). One

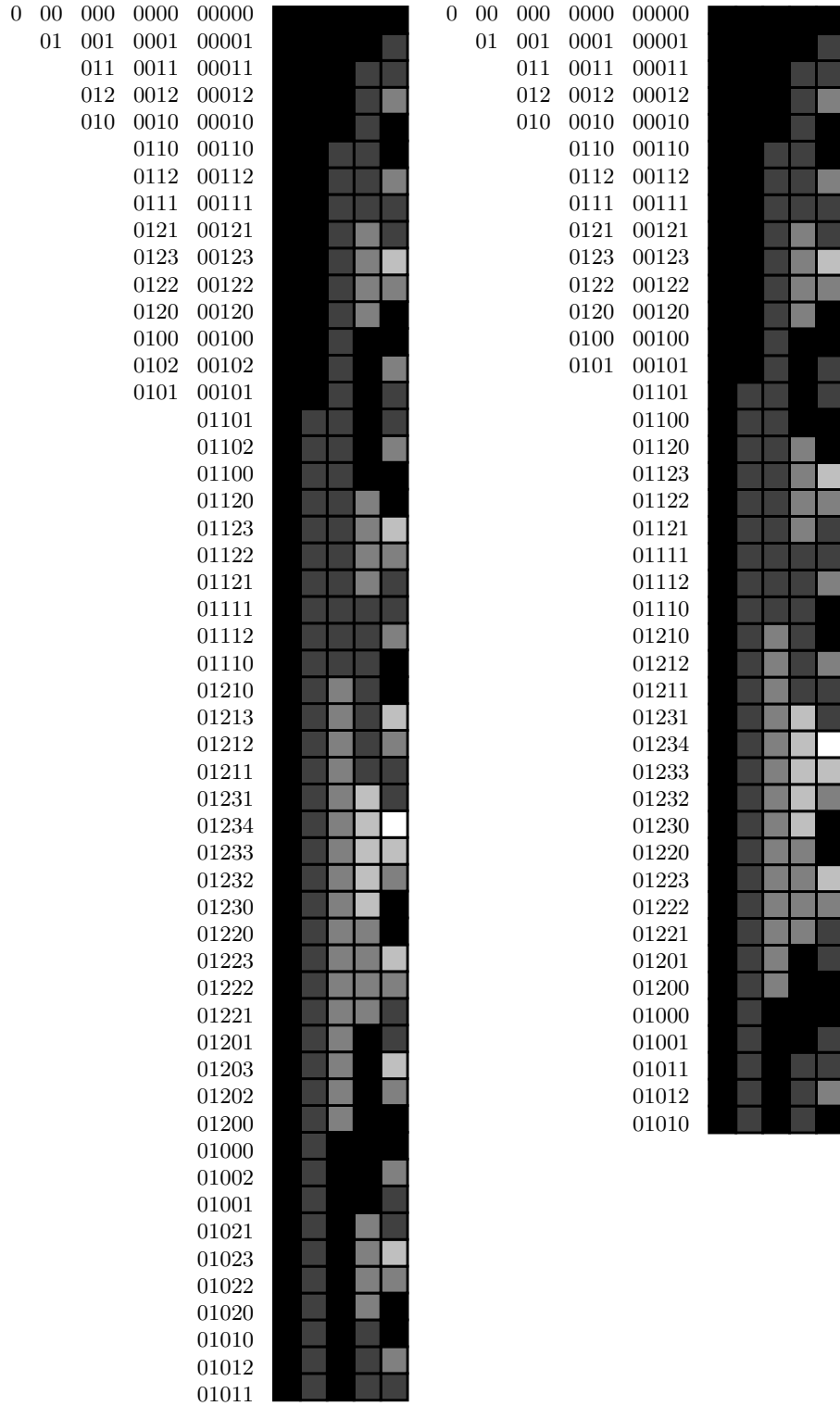


Fig. 3: Gray codes obtained from our max-right algorithm: start from 0^n then greedily maximize the rightmost possible digit.

Algorithm 1 Loopless algorithms for generating mixed-radix Gray codes. The functions modify our target $\mathbf{a}$ and yield it every time it is modified. Focus pointers are stored in $\mathbf{f}$. In **MixedRadix**, a_i is modified by $+1$ or -1 depending on the direction given by $\mathbf{d}$. In **MixedRadixMax**, as discussed in Section 3.3, any position has 0 and 1 as the first and last value (or vice versa) in a loop. a_i is raised to maximum ($\mathbf{b}_j - 1$) when $a_i = s_i$ (except in some special cases), and is decreased otherwise (normally it decreases by 1, but decreases by 2 if it is 2 and the start value is 1, in this case it needs to become 0). The overall algorithms are loopless as each iteration runs in worst-case $\mathcal{O}(1)$ -time.

MixedRadix(n)	MixedRadixMax(n)
1: $a_1 a_2 \cdots a_n \leftarrow 0 0 \cdots 0$	1: $a_1 a_2 \cdots a_n \leftarrow 0 0 \cdots 0$
2: $f_1 f_2 \cdots f_{n+1} \leftarrow 1 2 \cdots n+1$	2: $f_1 f_2 \cdots f_{n+1} \leftarrow 1 2 \cdots n+1$
3: $d_1 d_2 \cdots d_n \leftarrow 1 1 \cdots 1$	3: $s_1 s_2 \cdots s_n \leftarrow 0 0 \cdots 0$
4: while $f_1 \leq n$	4: while $f_1 \leq n$
5: $j \leftarrow f_1$	5: $j \leftarrow f_1$
6: $f_1 \leftarrow 1$	6: $f_1 \leftarrow 1$
7: $a_j \leftarrow a_j + d_j$	7: if $a_j = s_j$
8: yield $j, \mathbf{a}$	8: if $b_j = 2$ and $s_j = 1$
9: if $a_j \in b_j - 1, 0$	9: $a_j \leftarrow 0$
10: $d_j \leftarrow -d_j$	10: else
11: $f_j \leftarrow f_{j+1}$	11: $a_j \leftarrow b_j - 1$
12: $f_{j+1} \leftarrow j + 1$	12: else if $a_j = 2$ and $s_j = 1$
	13: $a_j \leftarrow 0$
	14: else
	15: $a_j \leftarrow 1$
	16: yield $j, \mathbf{a}$
	17: if $a_j = 1 - s_j$
	18: $s_j \leftarrow a_j$
	19: $f_j \leftarrow f_{j+1}$
	20: $f_{j+1} \leftarrow j + 1$

exception is that these digits are all 0s precisely when the digit is being changed for the first time. To track this special case, we store whether or not a digit has ever been changed in a Boolean list $\mathbf{v}$; Collectively, this additional information allows us to determine the maximum value for a digit in worst-case $\mathcal{O}(1)$ -time.

Theorem 5. *BellStrings in Algorithm 2 looplessly generates the max-right Gray code for Bell strings.*

6 Final Remarks

We provided Gray codes for restricted growth strings parameterized by $(s, f, \mathbf{c})$. The orders change one digit by -1 or -2 (cyclically) and are generated starting from 0^n by a simple greedy rule: set the rightmost possible digit to the maximum possible value that gives a new string. Our greedy max-right algorithms are not

Algorithm 2 Loopless algorithms for generating k -Catalan Gray codes and Bell Gray codes. The functions modify our target $\mathbf{a}$ and yield it every time it is modified. Focus pointers are stored in $\mathbf{f}$. **CatalanStrings** largely replicates **MixedRadixMax**, except that the “bases” are calculated on the fly. In **BellStrings**, if the current digit is not visited, the maximum is set to 0 because all earlier digits are 0. If it is visited and the stack storing positions of large numbers is empty, the maximum is set to 1. If the stack is not empty, the maximum is set to the corresponding digit at the position determined by the top of stack. After calculating the maximum, it will be pushed onto the stack. When a digit is decreased, if it corresponds to the top of stack, the stack is popped.

CatalanStrings(n, k)	BellStrings(n)
1: $a_1 a_2 \cdots a_n \leftarrow 0\ 0 \cdots 0$	1: $a_1 a_2 \cdots a_n \leftarrow 0\ 0 \cdots 0$
2: $f_1 f_2 \cdots f_{n+1} \leftarrow 1\ 2 \cdots n+1$	2: $f_1 f_2 \cdots f_{n+1} \leftarrow 1\ 2 \cdots n+1$
3: $s_1 s_2 \cdots s_n \leftarrow 0\ 0 \cdots 0$	3: $s_1 s_2 \cdots s_n \leftarrow 0\ 0 \cdots 0$
4: yield $j, \mathbf{a}$	4: $S \leftarrow \text{empty}$
5: while $f_1 < n$	5: $v_1 \cdots v_n \leftarrow \text{True} \cdots \text{True}$
6: $j \leftarrow f_1$	6: yield $j, \mathbf{a}$
7: $f_1 \leftarrow 1$	7: while $f_1 < n$
8: if $a_j = s_j$	8: $j \leftarrow f_1$
9: if $a_j, a_{j+1} = 1, 0$ and $k = 2$	9: $f_1 \leftarrow 1$
10: $a_j \leftarrow 0$	10: if $a_j = s_j$
11: else	11: if $v_j = \text{True}$
12: $a_j \leftarrow a_{j+1} + k - 1$	12: $m \leftarrow 0$
13: else if $a_j = 2$ and $s_j = 1$	13: $v_j \leftarrow \text{False}$
14: $a_j \leftarrow 0$	14: else if S is empty
15: else	15: $m \leftarrow 1$
16: $a_j \leftarrow -1$	16: else
17: yield $j, \mathbf{a}$	17: $pos \leftarrow \text{top}(S)$
18: if $a_j = 1 - s_j$	18: $m \leftarrow a_{pos}$
19: $s_j \leftarrow a_j$	19: else if $a_j = 2$ and $s_j = 1$
20: $f_j \leftarrow f_{j+1}$	20: $a_j \leftarrow 0$
21: $f_{j+1} \leftarrow j + 1$	21: if $\text{top}(S) = j$
	22: $\text{pop}(S)$
	23: else
	24: $a_j \leftarrow -1$
	25: if $\text{top}(S) = j$
	26: $\text{pop}(S)$
	27: yield $j, \mathbf{a}$
	28: if $a_j = 1 - s_j$
	29: $s_j \leftarrow a_j$
	30: $f_j \leftarrow f_{j+1}$
	31: $f_{j+1} \leftarrow j + 1$

efficient, but the orders can be efficiently generated by other means. We showed this with loopless algorithms for mixed-radix strings, k -Catalan strings, and

Bell strings. In particular, the k -Catalan strings appear to be new within the literature.

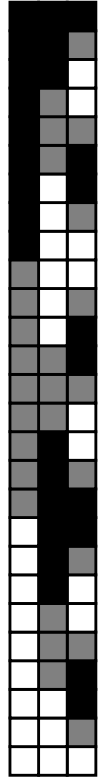
References

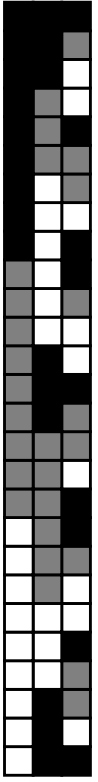
1. Arndt, J.: *Matters Computational: ideas, algorithms, source code*. Springer Science & Business Media (2010)
2. Cardinal, J., Hoang, H.P., Merino, A., Mička, O., Mütze, T.: Combinatorial generation via permutation languages. v. acyclic orientations. *SIAM Journal on Discrete Mathematics* **37**(3), 1509–1547 (2023)
3. Cardinal, J., Merino, A., Mütze, T.: Efficient generation of elimination trees and graph associahedra. In: *Proceedings of the 2022 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. pp. 2128–2140. SIAM (2022)
4. Ehrlich, G.: Loopless algorithms for generating permutations, combinations, and other combinatorial configurations. *Journal of the ACM (JACM)* **20**(3), 500–513 (1973)
5. Gray, F.: Pulse code communication. United States Patent Number 2632058 (1953)
6. Hartung, E., Hoang, H., Mütze, T., Williams, A.: Combinatorial generation via permutation languages. i. fundamentals. *Transactions of the American Mathematical Society* **375**(4), 2255–2291 (2022)
7. Hoang, H.P., Mütze, T.: Combinatorial generation via permutation languages. II. lattice congruences. *Israel Journal of Mathematics* **244**(1), 359–417 (2021)
8. Inc., O.F.: Bell or exponential numbers, Entry A000110 in the *On-line Encyclopedia of Integer Sequences*. <https://oeis.org/A000110> (2023), accessed on August 30 2023
9. Kaye, R.: A Gray code for set partitions. *Information Processing Letters* **5**, 171–173 (1976)
10. Kerr, K.: Successor rule for a restricted growth string gray code. unpublished manuscript (2015)
11. Knuth, D.E.: *The art of computer programming, volume 4A: combinatorial algorithms, part 1*. Pearson Education India (2011)
12. Knuth, D.E.: *The art of computer programming: generating all tuples and permutations*. Addison-Wesley. (2005)
13. Li, Y., Sawada, J.: Gray codes for reflectable languages. *Information processing letters* **109**(5), 296–300 (2009)
14. Mansour, T., Nassar, G., Vajnovszki, V.: Loop-free Gray code algorithm for the e-restricted growth functions. *Information Processing Letters* **111**(11), 541–544 (2011)
15. Mansour, T., Vajnovszki, V.: Efficient generation of restricted growth words. *Information Processing Letters* **113**(17), 613–616 (2013)
16. Merino, A., Mütze, T.: Combinatorial generation via permutation languages. III. rectangulations. *Discrete & Computational Geometry* pp. 1–72 (2022)
17. Mütze, T.: Combinatorial Gray codes-an updated survey. *The Electronic Journal of Combinatorics* **30**(3), DS26 (2022)
18. Ruskey, F.: *Combinatorial generation*. Preliminary working draft. University of Victoria, Victoria, BC, Canada **11**, 20 (2003)
19. Ruskey, F., Savage, C.D.: Gray codes for set partitions and restricted growth tails. *Australas. J Comb.* **10**, 85–96 (1994)
20. Sabri, A., Vajnovszki, V.: Two reflected Gray code-based orders on some restricted growth sequences. *The Computer Journal* **58**(5), 1099–1111 (2015)
21. Savage, C.: A survey of combinatorial Gray codes. *SIAM review* **39**(4), 605–629 (1997)

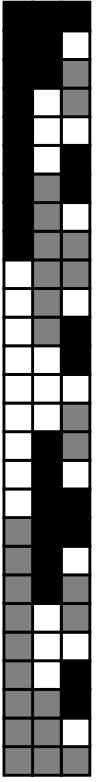
22. Stanley, R.P.: Enumerative combinatorics volume 1 second edition. Cambridge studies in advanced mathematics (2011)
23. Stanley, R.P.: Catalan numbers. Cambridge University Press (2015)
24. Stedman, F.: Campanalogia: or the Art of Ringing Improved, With plain and easie rules to guide the Practitioner in the Ringing all kinds of Changes, To Which is added, great variety of New Peals. London (1677)
25. Williams, A.: The greedy Gray code algorithm. In: Algorithms and Data Structures: 13th International Symposium, WADS 2013, London, ON, Canada, August 12-14, 2013. Proceedings 13. pp. 525–536. Springer (2013)
26. Zaks, S.: Lexicographic generation of ordered trees. Theoretical Computer Science **10**(1), 63–82 (1980)

A Ternary String Gray Codes

Figure 4 illustrates the three greedy approaches to generating ternary strings. (The first approach discussed in Section 3.2 does not generalize to ternary strings since there is no natural notion of complementation in this context.)

tgc(3, $\pm$)	$a_3a_2a_1$	$\pm$
	000	+3
	001	+3
	002	+2
	012	-3
	011	-3
	010	+2
	020	+3
	021	+3
	022	+1
	122	-3
	121	-3
	120	-2
	110	+3
	111	+3
	112	-2
	102	-3
	101	-3
	100	+1
	200	+3
	201	+3
	202	+2
	212	-3
	211	-3
	210	+2
	220	+3
	221	+3
	222	

tgc(3, $\oplus$)	$a_3a_2a_1$	$\oplus$
	000	$\oplus_3$
	001	$\oplus_3$
	002	$\oplus_2$
	012	$\oplus_3$
	010	$\oplus_3$
	011	$\oplus_2$
	021	$\oplus_3$
	022	$\oplus_3$
	020	$\oplus_1$
	120	$\oplus_3$
	121	$\oplus_3$
	122	$\oplus_2$
	102	$\oplus_3$
	100	$\oplus_3$
	101	$\oplus_2$
	111	$\oplus_3$
	112	$\oplus_3$
	110	$\oplus_1$
	210	$\oplus_3$
	211	$\oplus_3$
	212	$\oplus_2$
	222	$\oplus_3$
	220	$\oplus_3$
	221	$\oplus_2$
	201	$\oplus_3$
	202	$\oplus_3$
	200	

tgc(3, max)	$a_3a_2a_1$	max
	000	max ₃
	002	max ₃
	001	max ₂
	021	max ₃
	022	max ₃
	020	max ₂
	010	max ₃
	012	max ₃
	011	max ₁
	211	max ₃
	212	max ₃
	210	max ₂
	220	max ₃
	222	max ₃
	221	max ₂
	201	max ₃
	202	max ₃
	200	max ₁
	100	max ₃
	102	max ₃
	101	max ₂
	121	max ₃
	122	max ₃
	120	max ₂
	110	max ₃
	112	max ₃
	111	

(a) Increment/decrement.

(b) Cyclic increment.

(c) Maximize digit.

Fig. 4: Three greedy Gray codes for ternary strings. Each one greedily applies an operation (or operations) to the rightmost possible digit.

B Python Code

Python implementations of the loopless algorithms in Section 5 are provided.

```

def mixedRadixGrayCodeMax(bases):
    n = len(bases)
    word = [0] * n
    start = [0] * n
    yield word, None
    focus = list(range(n+1))
    while focus[0] < n:
        index = focus[0]
        focus[0] = 0
        if word[index] == start[index]:
            if bases[index] == 2 and start[index] == 1:
                word[index] = 0 # special case of start == max
            else:
                word[index] = bases[index]-1 # set to max
        elif word[index] == 2 and start[index] == 1:
            word[index] -= 2
        else:
            word[index] -= 1
        yield word, index
        if word[index] == 1-start[index]:
            start[index] = word[index]
            focus[index] = focus[index+1]
            focus[index+1] = index+1

bases = [2,3,4]
total = 0
for word, change in mixedRadixGrayCodeMax(bases):
    total += 1
    print(*word, sep="", end=" ")
    print(change)
print("\ntotal: %d / %d" % (total, prod(bases)))

```

```

def looplessCatalanStrings(n):
    word = [0] * n
    yield word
    focus = list(range(n+1))
    start = [0] * n
    while focus[0] < n-1:
        index = focus[0]
        focus[0] = 0
        if word[index] == start[index]:
            # set to max
            # but handle special case where it is both the start and the max already
            if word[index] == 1 and word[index+1] == 0:
                word[index] = 0
            else:
                word[index] = word[index+1]+1
        elif word[index] == 2 and start[index] == 1:
            # skip over 1
            word[index] -= 2
        else:
            word[index] -= 1
        yield word
        if word[index] + start[index] == 1: # last value (i.e., 0+1 or 1+0)
            focus[index] = focus[index+1]
            focus[index+1] = index+1
            start[index] = word[index]

n = 5
total = 0
for word in looplessCatalanStrings(n):
    total += 1
    print(*word, sep=" ")
print("\ntotal: %d" % total)

```

```

def looplessBellStrings(n):
    word = [0] * n
    yield word
    focus = list(range(n+1))
    start = [0] * n
    maxima = [] # indices that create maxima values ...,3,2
    first = [True] * n # first if the digit hasn't been changed yet

    while focus[0] < n-1:
        # maximaValues = [word[index] for index in maxima]
        index = focus[0]
        focus[0] = 0
        if word[index] == start[index]:
            # set to max
            if first[index]: # only time when digits to the right are all 0
                m = 0
                first[index] = False
                assert len(maxima) == 0
            elif len(maxima) == 0: # no 2s and not first so max is 1
                m = 1
            else:
                m = word[maxima[0]]

            word[index] = m+1
            if m+1 != 1:
                maxima = [index] + maxima

        elif word[index] == 2 and start[index] == 1:
            # skip over 1
            word[index] -= 2
            if maxima[0] == index:
                maxima = maxima[1:]
            else:
                word[index] -= 1
                if maxima[0] == index:
                    maxima = maxima[1:]
        yield word

        if word[index] + start[index] == 1: # last value (i.e., 0+1 or 1+0)
            focus[index] = focus[index+1]
            focus[index+1] = index+1
            start[index] = word[index]

n = 7
total = 0
for word in looplessBellStrings(n):
    total += 1
    print(*word, sep=" ")
print("\ntotal: %d" % total)

```